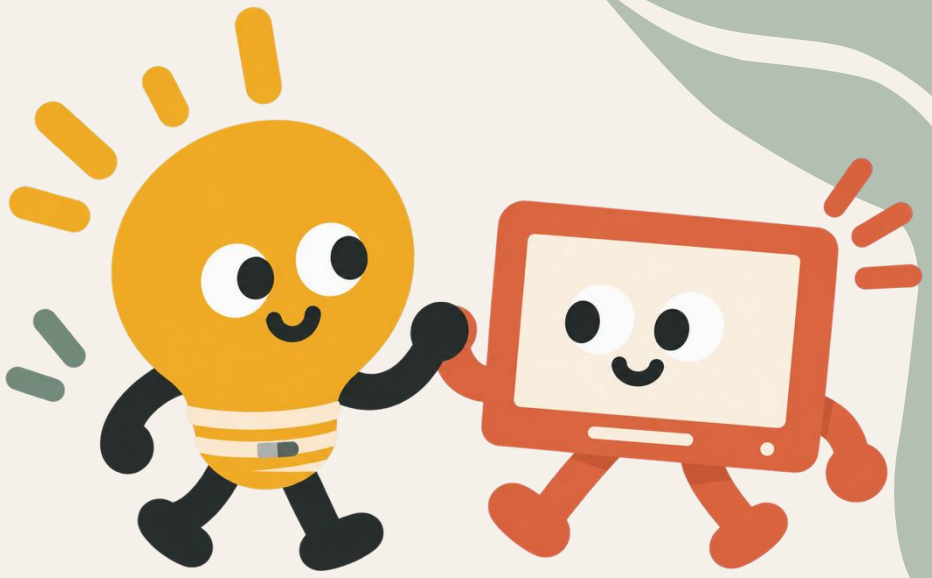
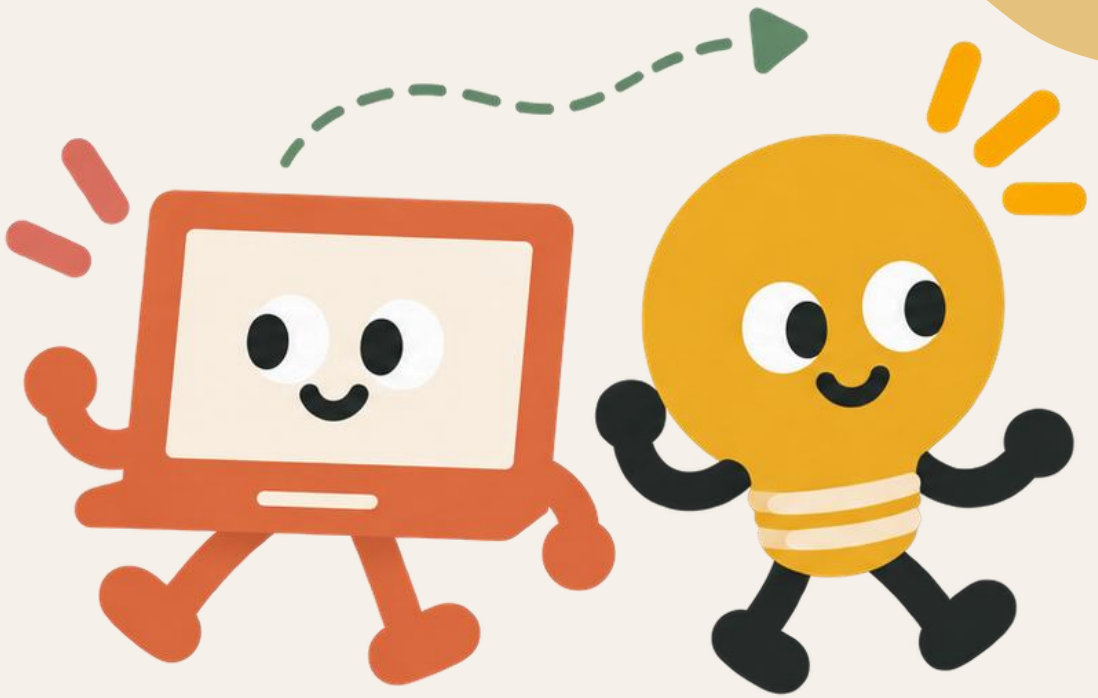
Módulo 5

Taller teórico-práctico
Del dataset a U-Net: segmentación de imágenes con Deep Learning

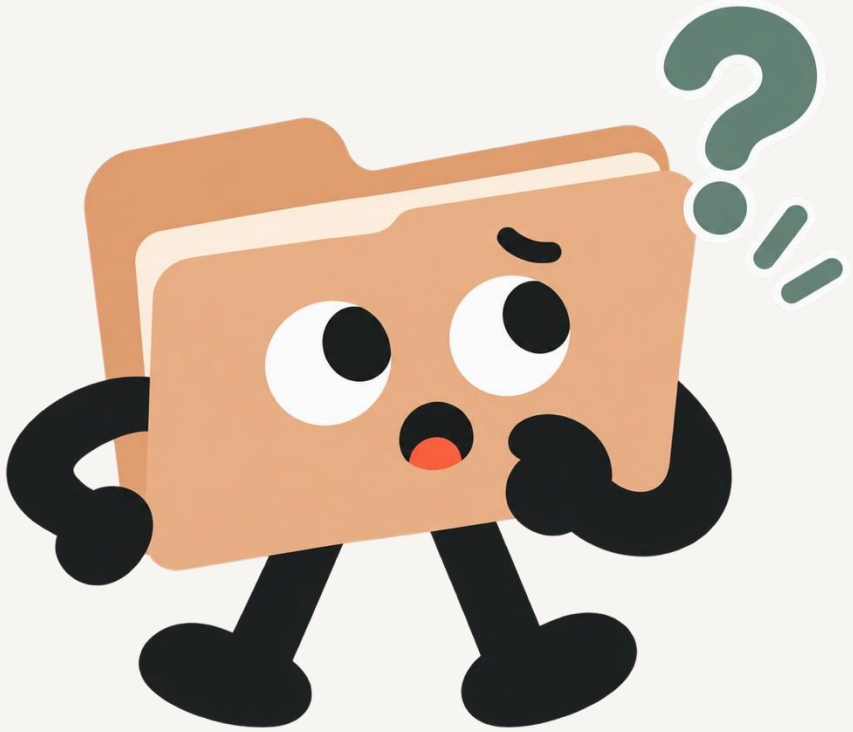
→ De la convolución a U-Net



Arquitectura U-Net



¿Cómo puede una red neuronal comprender qué aparece en una imagen y, al mismo tiempo, conservar la información espacial necesaria para localizarlo píxel por píxel?



Arquitectura Encoder–Decoder

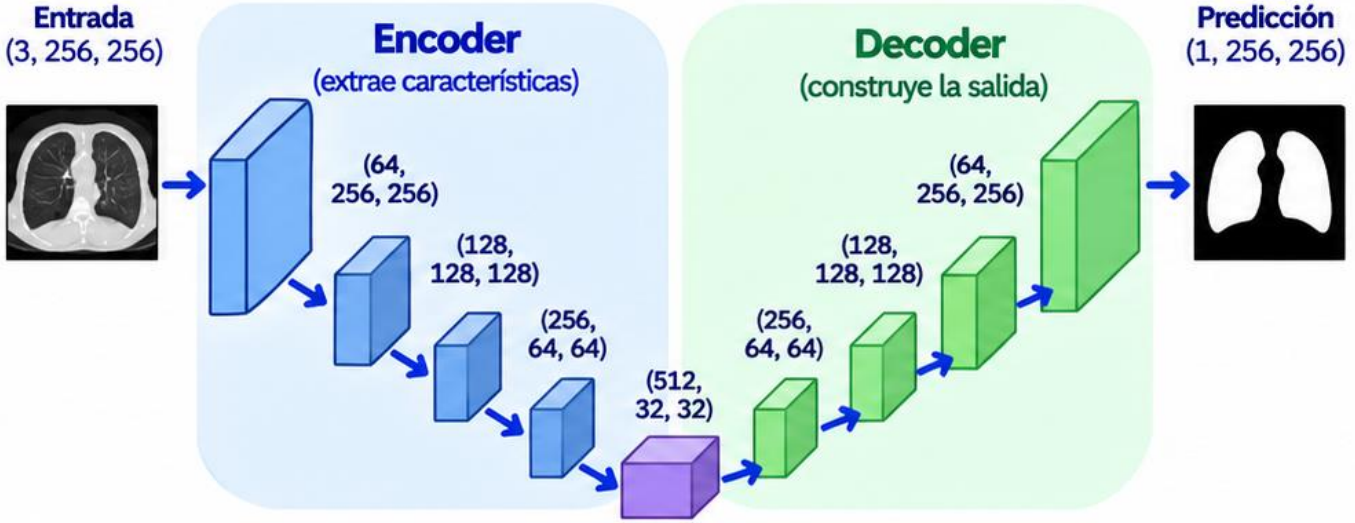
Procesa la imagen en dos etapas complementarias: extrae características y construye una representación de salida.

1 Encoder (codificador)

- Extrae características de la imagen mediante convoluciones.
- A medida que avanza por sus niveles, disminuye la resolución espacial ($H \times W$) y aumenta el número de canales (C).
- Aprende representaciones progresivamente más abstractas: desde bordes y texturas hasta patrones y estructuras complejas.



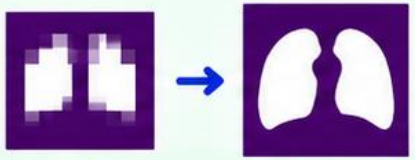
Encoder–Decoder



Evolución de las dimensiones (típico, p. ej. en U-Net)	Entrada	↓ Encoder (menor resolución, más canales)	Cuello (representación más abstracta)	↑ Decoder (mayor resolución, menos canales)	Salida (máscara de segmentación)
	(3, 256, 256) →	(64, 256, 256) (128, 128, 128) (256, 64, 64) →	(512, 32, 32) →	(256, 64, 64) (128, 128, 128) (64, 256, 256) →	(1, 256, 256)

2 Decoder (decodificador)

- Utiliza las características obtenidas por el encoder para construir la salida.
- Incrementa progresivamente las dimensiones espaciales mediante upsampling o convoluciones transpuestas.
- En segmentación semántica, genera una predicción que asigna una clase a cada píxel de la imagen.



Resumen

- **Encoder:** extrae características, ↓ resolución, ↑ canales.
- **Decoder:** construye la salida, ↑ resolución, ↓ canales.
- En conjunto, permiten transformar una imagen de entrada en una representación de salida (p. ej., una máscara de segmentación).

U-Net: Skip Connections y Concatenación

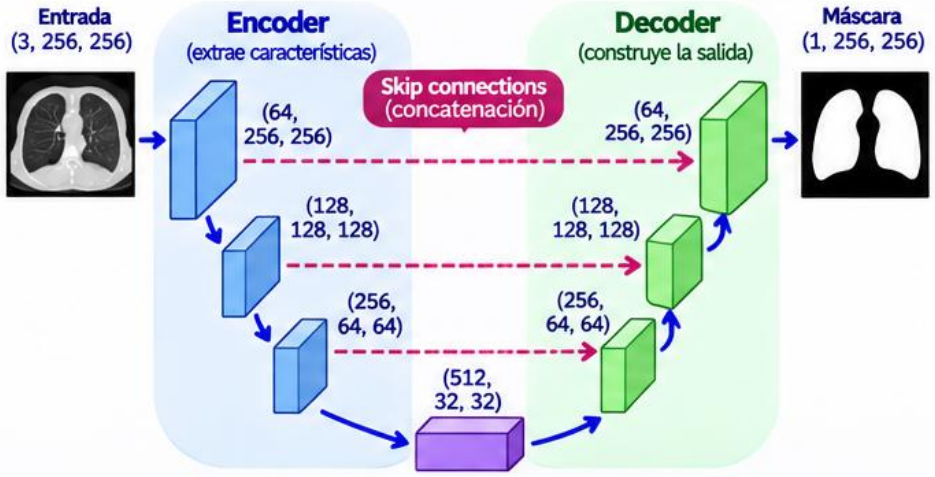
Permiten combinar la información espacial del encoder con las características procesadas del decoder para producir máscaras de segmentación con más detalles.

1 ¿Qué es una skip connection?

- Conexión que transfiere directamente mapas de características del encoder al decoder, sin pasar por todas las capas intermedias.
- En U-Net, permite reutilizar información espacial de mayor resolución obtenida antes de las reducciones (p. ej., pooling).
- Su función es preservar detalles espaciales y mejorar la localización de los objetos.

Importante: no recupera información perdida por el pooling; permite reutilizar la información que se conservó antes de esa reducción.

U-Net con skip connections (concatenación)



2 Concatenación de mapas de características

- Une dos tensores a lo largo de la dimensión de canales (dim = 1).
- Conserva el batch, la altura y la anchura; solo se suman los canales.

$$(N, C_1, H, W) + (N, C_2, H, W) \rightarrow (N, C_1 + C_2, H, W)$$

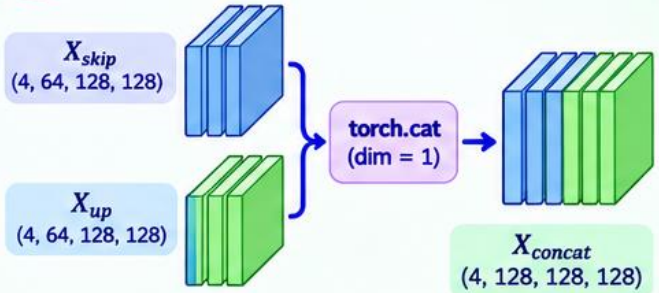
Ejemplo:

$X_{skip} \in \mathbb{R}^{4\times64\times128\times128}$

$X_{up} \in \mathbb{R}^{4\times64\times128\times128}$

$\rightarrow X_{concat} \in \mathbb{R}^{4\times128\times128\times128}$

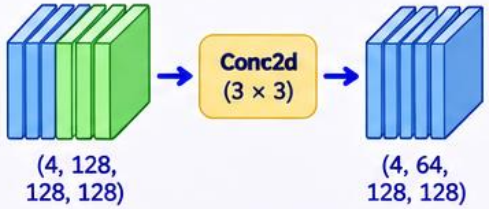
3 Implementación en PyTorch



torch.cat une los tensores. No tiene parámetros entrenables (0 parámetros).

4 ¿Qué sucede después?

El tensor concatenado se procesa con nuevas convoluciones, que aprenden a combinar la información de ambas rutas y, normalmente, reducen el número de canales.



5 Concatenación vs. suma

Característica	Concatenación (torch.cat)	Suma (+)
Operación	Une mapas de características	Suma valores elemento a elemento
Número de canales	Se suman ($C_1 + C_2$)	Se conserva (C)
Dimensiones requeridas	Deben coincidir salvo en el eje de concatenación	Deben ser compatibles para la suma
Parámetros entrenables	0	0
Uso característico	U-Net	Bloques residuales de ResNet

Ejemplo: (4, 64, 128, 128) + (4, 64, 128, 128)

- **Concatenación:** (4, 128, 128, 128)
- **Suma:** (4, 64, 128, 128)

Resumen



- **Skip connections:** transfieren mapas del encoder al decoder, preservando información espacial.
- **Concatenación:** une los mapas de características por canales, conservando las dimensiones espaciales.
- **En conjunto:** permiten que el decoder combine contexto (niveles profundos) con detalles espaciales (niveles anteriores), logrando una segmentación más precisa y con límites mejor definidos.

U-Net: Capa de salida y predicción por píxel

Convierte los mapas de características del decoder en una máscara de segmentación.

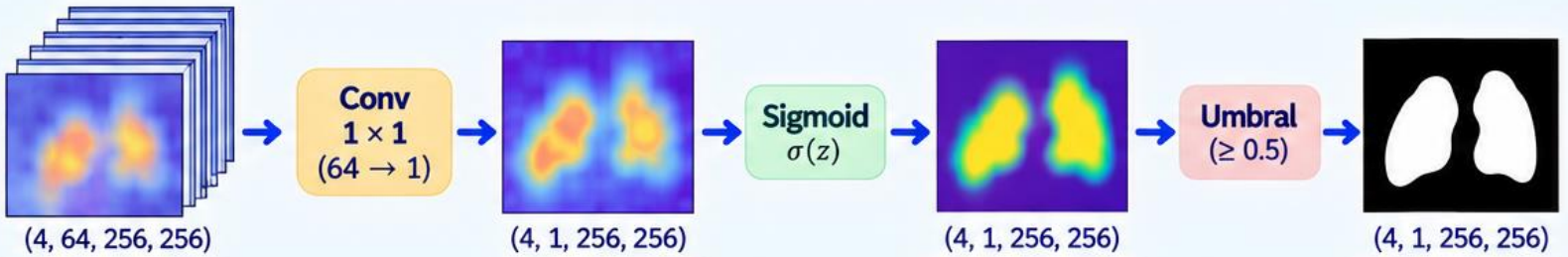
Contexto: salida del decoder

El último bloque del decoder produce un tensor con características espaciales y semánticas:

$$X \in \mathbb{R}^{4\times64\times256\times256}$$

- 4 imágenes (batch)
- **64** canales (mapas de características)
- Resolución: 256 × 256

De mapas de características a máscara de segmentación (binaria)



1 Convolución de 1 × 1

Combina los 64 canales de cada posición espacial para producir **un logit** por píxel, sin cambiar la altura ni la anchura.

$$z_{h,w} = \sum_{c=1}^{64} w_c X_{c,h,w} + b$$

- w_c: peso aprendido para el canal c.
- b: sesgo aprendido.
- $z_{h,w}$: logit (valor sin normalizar).

Parámetros: $64(1)(1)(1) + 1 = \mathbf{65}$

2 De logit a máscara (binaria)

Se aplica la función **Sigmoid** para obtener probabilidades y luego un umbral.

$$p_{h,w} = \sigma(z_{h,w}) = \frac{1}{1+e^{-z_{h,w}}}$$

$$\hat{y}_{h,w} = \begin{cases} 1, & p_{h,w} \geq 0.5 \\ 0, & p_{h,w} < 0.5 \end{cases}$$

Ejemplo (4 × 4)

0.08	0.16	0.34	0.12
0.19	0.78	0.91	0.24
0.13	0.86	0.97	0.68
0.05	0.28	0.73	0.18

Mapa de probabilidades $(p_{h,w})$

→

0	0	0	0
0	1	1	0
0	1	1	1
0	0	1	0

Máscara (umbral = 0.5)

3 Segmentación multiclase

Para K clases, la capa final produce K canales:

`Conv2d(64, K, kernel_size = 1)`

- Salida: (N, K, H, W)
- Se aplica Softmax para obtener probabilidades por clase.
- La clase predicha se selecciona con argmax(dim = 1).



(4, 3, 256, 256) (probabilidades) → (4, 256, 256) (etiquetas de clase)

4 Binaria vs. multiclase

Característica	Binaria	Multiclase
Canales de salida	1	K (número de clases)
Activación (por píxel)	Sigmoid	Softmax
Predicción final	Umbral (p. ej., 0.5)	Argmax (dim = 1)
Función de pérdida (con logits)	BCEWithLogitsLoss	CrossEntropyLoss

ⓘ Las funciones de pérdida reciben los logits directamente (sin aplicar *Sigmoid* ni *Softmax*).

Resumen



- El decoder produce mapas de características con información espacial y semántica.
- Una convolución de 1 ×1 combina los canales para generar un logit por clase y por píxel.
- Una función de activación (Sigmoid o Softmax) convierte los logits en probabilidades, y un umbral o argmax produce la máscara de segmentación.

La red no dibuja directamente una máscara: aprende a realizar una clasificación para cada píxel de la imagen.